

Take-Home Exercise

Multi-Agent GenAI Quote Advisor

AI Tech Lead Candidate Exercise · Expected effort: 6–10 hours

Objective

Build a multi-agent system that generates a structured home insurance quote recommendation from a customer profile. The goal is to demonstrate your ability to design and implement a system that uses LLMs in a controlled, reliable, and extensible way, with clear separation between reasoning, tooling, and orchestration.

Problem Statement

Build a system that:

- Accepts a customer profile as structured JSON input
- Uses multiple agents with distinct responsibilities to assess risk, recommend coverage, and estimate a premium
- Provides agents with deterministic tools that handle structured computation or context lookup, separate from LLM reasoning
- Produces a structured JSON output with explanation
- Supports a follow-up question in the same session, such as: "Why is this quote expensive?"

Key Requirements

1. Multi-Agent Design

Define multiple agents with clear, non-overlapping responsibilities. We are looking for meaningful architectural decomposition, not just multiple prompt calls wrapped in functions.

Suggested agent roles (you may adapt these):

- **Risk Assessment Agent:** Evaluates the customer profile and identifies risk factors.
- **Coverage Recommendation Agent:** Maps risk factors to appropriate coverage types and limits.
- **Pricing Agent:** Estimates a premium range based on risk and coverage selections.
- **Validator Agent (optional but encouraged):** Checks output consistency across agents before final output is produced.

What we mean by "meaningful decomposition":

- Each agent has its own prompt context, responsibility boundary, and output contract

- Agents communicate through defined inputs/outputs, not by sharing a monolithic prompt
- The orchestrator coordinates agents rather than one agent calling another ad hoc

2. Agent Tools

Implement at least two tools that agents can invoke. Tools represent deterministic, reusable computations or data lookups, they are explicitly separated from LLM reasoning.

Each tool must:

- Accept structured input (e.g., a typed dict or Pydantic model)
- Return structured output
- Contain no LLM calls, tools are deterministic and independently testable
- Be reusable across agents if the use case arises

Example tools:

- **RiskFactorTool:** Scores individual risk dimensions (e.g., location hazard tier, pool liability, claims history) and returns a risk breakdown.
- **PricingHeuristicTool:** Applies actuarial-style multipliers to a base premium using structured rules, no LLM involved.
- **CoverageRulesTool:** Returns eligible coverage types and statutory minimums based on location and home value.

Note on terminology: In this exercise, "tool" means a structured, callable function, consistent with how tools are defined in the OpenAI and Anthropic function-calling APIs. You are not required to implement the MCP wire protocol. If you are familiar with MCP and choose to use it, document your decision.

3. Workflow and Orchestration

Define how agents are sequenced and how state flows between them. Your system should have a clear controller or orchestrator layer, not just a top-level script that calls agents in order.

We expect to see:

- Explicit sequencing logic with dependency handling (e.g., pricing depends on coverage selection, which depends on risk)
- A mechanism for passing outputs from one agent as inputs to the next
- The orchestrator as a distinct layer that agents do not need to be aware of

You may use a framework (LangGraph, LlamaIndex Workflows, CrewAI, or a custom orchestrator over the Anthropic or OpenAI SDK). If you use a framework, briefly explain why you chose it and what it gives you over a hand-rolled approach.

4. Structured Reasoning

Your system should demonstrate that it breaks the problem into meaningful intermediate steps rather than producing the final answer in a single monolithic prompt.

This can include:

- Intermediate outputs that are passed between agents (e.g., a risk score object used by the pricing agent)
- Chain-of-thought or scratchpad reasoning within a single agent before producing its structured output
- Explicit evidence linking, the explanation in the final output should trace back to specific risk factors or coverage decisions

5. Handling Non-Determinism

LLM outputs are inherently non-deterministic. Your system must explicitly address this, not just acknowledge it.

Choose and implement at least one approach:

- **Output normalization:** Post-process LLM output to enforce consistent formats (e.g., always normalize coverage names to a controlled vocabulary).
- **Retry with validation:** If structured output fails schema validation, retry the call and log the failure.
- **Consensus / voting:** Run the same agent prompt N times and select the majority output or average numeric values. (Note: this is the most costly approach, scope it carefully given the time budget.)
- **Deterministic post-processing:** Use rule-based logic after LLM output to clamp values, fill gaps, or override implausible results.

Document your choice clearly in the README: what you implemented, why you chose it, and what its limitations are.

6. Guardrails

Implement at least two of the following:

- **JSON schema validation:** Validate agent outputs against a defined schema before passing them downstream.
- **Input validation:** Reject or flag malformed or incomplete customer profiles before processing begins.
- **Output consistency checks:** Verify that final output fields are internally consistent (e.g., premium range aligns with identified risk level).
- **Fallback handling:** Define explicit behavior when an agent is uncertain or returns an invalid output (e.g., return a conservative default with a low confidence score).
- **Deterministic output formatting:** Ensure field names, value ranges, and enumerations are normalized regardless of LLM phrasing.

Note on confidence score: The required output includes a `confidence_score` field. You must define and document how this is computed, it should not be a raw LLM self-assessment. Acceptable approaches include: averaging validation pass rates across agents, a heuristic based on the number of guardrail checks passed, or a combination of rule-based signals. Explain your method in the README.

Input / Output Contracts

Sample Customer Profiles

Use both profiles in your testing. The second profile is intentionally incomplete, your input validation guardrail should handle it.

Profile A, Complete input

```
{ "age": 50, "location": "California", "home_value": 900000,
  "has_pool": true, "claims_history": 1, "credit_score": 700 }
```

Profile B, Incomplete / edge-case input

```
{ "age": 34, "location": "Florida", "home_value": 450000,
  "has_pool": false, "claims_history": 0, "credit_score": null }
```

Your system should document how it handles the missing `credit_score`, whether it applies a conservative default, requests the value, or flags low confidence.

Expected Output Schema

```
{
  "risk_factors": [
    { "factor": "string", "severity": "low | medium | high", "rationale":
"string" }
  ],
  "recommended_coverages": [
    { "type": "string", "limit": "string", "rationale": "string" }
  ],
  "premium_range": { "low": number, "high": number, "currency": "USD" },
  "explanation": "string",
  "confidence_score": number, // 0.0 - 1.0, method documented in README
  "warnings": ["string"]      // e.g., missing fields, capped values
}
```

Follow-Up Question Handling

Your CLI must support at least one follow-up question after the initial quote is generated, such as:

- "Why is this quote expensive?"
- "What would happen if I removed the pool?"

This tests whether your architecture supports session context, the follow-up agent or handler must have access to the prior quote output and intermediate reasoning, not just the original customer profile.

You do not need to build a full conversational loop. A single structured follow-up exchange is sufficient.

Deliverables

Code

- GitHub repository (public or shared with reviewers)
- Clean, modular structure with clear separation of agents, tools, and orchestration
- A requirements file or equivalent (requirements.txt, pyproject.toml, package.json)
- No secrets or API keys committed, use environment variables

README

The README is a primary evaluation artifact. Include:

- **Architecture diagram:** A simple diagram (ASCII, Mermaid, or image) showing agent flow and tool interactions.
- **Agent descriptions:** What each agent does, its input/output contract, and its prompt strategy.
- **Tool descriptions:** What each tool computes and why it is deterministic.
- **Orchestration flow:** How the orchestrator sequences agents and handles dependencies.
- **Non-determinism approach:** What you implemented, why, and its limitations.
- **Guardrails:** Which guardrails you implemented and how they behave on invalid input.
- **Confidence score method:** How the score is computed.
- **Assumptions:** Any domain assumptions you made (e.g., pricing model basis, coverage taxonomy).
- **Trade-offs and improvements:** What you would do differently with more time.

CLI Demo

A runnable CLI that:

- Accepts a customer profile (JSON file or inline argument)
- Prints the structured output to stdout
- Supports at least one follow-up question prompt

Evaluation Criteria

Dimension	What We Are Looking For
Multi-agent design	Clear responsibilities, meaningful decomposition, not just wrapped prompts
Tool design	Deterministic, structured, testable, properly separated from LLM logic
Orchestration	Explicit sequencing, dependency handling, clean controller layer
Structured reasoning	Intermediate outputs, evidence linking, no monolithic prompts
Non-determinism handling	Implemented (not just described), with documented trade-offs
Guardrails	At least two implemented; edge-case input (Profile B) handled gracefully
Follow-up handling	Session context preserved; follow-up references prior reasoning
Code quality	Readable, modular, no hardcoded secrets
README clarity	Architecture explained, assumptions documented, trade-offs honest

Notes and Tips

- **Focus on clarity over completeness.** A clean, well-reasoned partial solution is preferable to a sprawling one that is hard to follow.
- **Document your assumptions.** Domain assumptions about pricing, coverage types, or risk scoring are expected, just state them.
- **Framework choice is yours.** We are not evaluating which framework you pick; we are evaluating how you use it and whether you understand the trade-offs.
- **Avoid over-engineering.** You do not need a production-grade system. You need a system that clearly demonstrates architectural thinking.
- **We will go deeper in the follow-up discussion.** Expect questions about design decisions, what you would change, and how the system would behave at scale.

Timeline

Please submit within one week of receiving this exercise. Expected effort is 6–10 hours.